Distributed Systems & Engineering 111-Topic Interview Playbook

For Technical Program Manager, Senior Engineering Manager, Principal TPM, and technofunctional engineering interviews.

Every topic includes: a practical definition, implementation/usage types, a real-world scenario, and interview follow-up questions.

Use this as a discussion guide, not a memorization sheet. For interviews, explain the trade-offs, failure modes, operational impact, and how you would measure success.

Quick Index

1. Rate Limiting
2. Caching
3. Load Balancing
4. Reverse Proxies
5. API Gateways
6. CI/CD
7. Docker
8. Kubernetes
9. Service Discovery
10. Circuit Breakers
11. Timeouts
12. Retries
13. Exponential Backoff
14. Idempotency
15. Message Queues
16. Pub/Sub
17. Event-Driven Architecture
18. Distributed Transactions
19. Saga Pattern
20. Dead Letter Queues
21. Cron Jobs
22. WebSockets
23. Long Polling
24. Server-Sent Events
25. Database Indexing
26. Query Optimization
27. N+1 Queries
28. Connection Pooling
29. Read Replicas
30. Sharding
31. Partitioning
32. Replication
33. Leader Election
34. CAP Theorem
35. Eventual Consistency
36. Optimistic Locking
37. Pessimistic Locking
38. Distributed Locks
39. Race Conditions
40. Deadlocks
41. Memory Leaks
42. Garbage Collection
43. Thread Safety
44. Backpressure
45. Autoscaling
46. Horizontal Scaling
47. Vertical Scaling
48. CDN
49. Edge Caching
50. Cache Invalidation
51. Feature Flags
52. Blue-Green Deployment
53. Canary Releases
54. Rolling Deployments
55. Rollbacks
56. Health Checks
57. Liveness & Readiness Probes
58. Monitoring
59. Logging
60. Distributed Tracing
61. Metrics
62. Alerts
63. SLOs
64. SLIs
65. Error Budgets
66. Observability
67. Secrets Management
68. IAM
69. OAuth
70. JWT Rotation
71. TLS
72. Encryption at Rest
73. Encryption in Transit
74. WAF
75. DDoS Protection
76. CORS
77. CSRF
78. SQL Injection
79. XSS
80. SSRF
81. Database Migrations
82. Schema Versioning
83. Disaster Recovery
84. Backups
85. Failover
86. Multi-Region Deployment
87. Chaos Engineering
88. Cost Optimization
89. Cold Starts
90. Serverless Limits
91. Latency
92. Throughput
93. P99 Latency
94. Tail Latency
95. Network Partitions
96. Clock Skew
97. DNS
98. TCP vs UDP
99. HTTP/2 & HTTP/3
100. gRPC
101. Webhooks
102. API Versioning
103. Semantic Versioning
104. Infrastructure as Code
105. Terraform
106. Helm Charts
107. Build Caching
108. Dependency Hell
109. Production Incidents
110. On-Call
111. Postmortems

1. Rate Limiting

Definition

Controls how many requests a client, user, token, or IP can make during a defined interval. It protects capacity, prevents abuse, and keeps one consumer from exhausting shared resources.

Implementation / Usage Types

- Fixed window: count requests in fixed intervals; simple but can allow bursts at boundaries.
- Sliding window: evaluates a rolling interval; smoother enforcement with more state.
- Token bucket: tokens refill at a rate and requests consume tokens; allows controlled bursts.
- Leaky bucket: processes traffic at a steady rate; useful when downstream work must be smoothed.

Real-World Example

An API serving 10,000 requests/second may allow each customer 100 requests/second. Excess requests receive HTTP 429 and a Retry-After hint.

Interview Follow-Up Questions

- Where should the limit be enforced: CDN, gateway, or service?
- How would you choose limits for different customer tiers?
- What happens when multiple application instances share the same limit?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

2. Caching

Definition

Stores reusable data closer to the consumer so future reads avoid expensive computation, database work, or network calls. The key design issues are freshness, eviction, capacity, and invalidation.

Implementation / Usage Types

- Client/browser cache: stores responses locally and reduces repeat network calls.
- CDN cache: serves cacheable content from edge locations near users.
- Distributed cache: Redis/Memcached-style shared cache for application data.
- Local in-process cache: fastest option, but each instance has its own copy.

Real-World Example

A product catalog service can cache frequently requested product details in Redis for 60 seconds, reducing database load during traffic spikes.

Interview Follow-Up Questions

- What is the acceptable staleness window?
- How do you handle cache stampedes?
- When would you choose Redis over an in-process cache?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

3. Load Balancing

Definition

Distributes incoming traffic across multiple healthy servers or service instances. It improves availability, capacity utilization, and horizontal scalability.

Implementation / Usage Types

- Layer 4: routes based on TCP/UDP connection information; fast and protocol-oriented.
- Layer 7: routes using HTTP attributes such as host, path, or headers.
- Round robin: distributes requests sequentially.
- Least connections: favors instances currently handling fewer connections.
- Weighted routing: sends more traffic to higher-capacity instances.

Real-World Example

An e-commerce API runs 20 application instances behind a load balancer. Health checks remove unhealthy instances, while weighted routing sends more traffic to larger nodes.

Interview Follow-Up Questions

- What is the difference between L4 and L7 balancing?
- How should sticky sessions affect scalability?
- What metrics determine whether the fleet needs more instances?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

4. Reverse Proxies

Definition

A server that sits in front of backend servers and receives client requests on their behalf. It can centralize TLS termination, routing, compression, caching, security, and access control.

Implementation / Usage Types

- TLS termination: decrypt HTTPS at the proxy and forward traffic internally.
- Path/host routing: send /api or different domains to different backends.
- Caching/compression: reduce repeated backend work and response size.
- Security gateway: hide origin servers and apply request filtering.

Real-World Example

NGINX can terminate HTTPS for a web application, route /api to a Node.js service, serve static assets directly, and forward dynamic requests to application servers.

Interview Follow-Up Questions

- What is the difference between a reverse proxy and a forward proxy?
- Where should TLS terminate?
- How do proxy timeouts influence incident behavior?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

5. API Gateways

Definition

Provides a controlled entry point for APIs, typically handling routing, authentication, authorization, rate limiting, transformations, observability, and policy enforcement.

Implementation / Usage Types

- Managed gateway: cloud/provider service handles much of the infrastructure.
- Self-hosted gateway: NGINX/Kong/Envoy-style deployment with greater control.
- API aggregation: combines several backend calls into one client-facing API.
- Policy gateway: centrally applies authentication, quotas, and request policies.

Real-World Example

A mobile banking app calls one gateway. The gateway validates the token, enforces customer-specific quotas, routes loan requests to the loan service, and emits audit telemetry.

Interview Follow-Up Questions

- What belongs in the gateway versus the microservice?
- How do you prevent the gateway becoming a bottleneck?
- How would you version gateway-managed APIs?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

6. CI/CD

Definition

Automates the path from source-code change to validated deployment. CI focuses on frequent integration and automated verification; CD extends this toward repeatable release and deployment.

Implementation / Usage Types

- Continuous Integration: build, lint, unit test, security scan, and package every change.
- Continuous Delivery: keep software deployable and require an explicit production release decision.
- Continuous Deployment: automatically release validated changes to production.
- Pipeline promotion: move the same immutable artifact through dev, staging, and production.

Real-World Example

A team merges a pull request, runs tests and dependency scans, builds a Docker image, deploys to staging, runs smoke tests, then promotes the same image to production.

Interview Follow-Up Questions

- Which checks are mandatory before production?
- How do you reduce pipeline duration without reducing confidence?
- How would you design safe rollback for a database-changing release?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

7. Docker

Definition

Packages an application and its dependencies into a portable container image. Containers isolate processes while sharing the host operating system kernel.

Implementation / Usage Types

- Single-process application container: package one service and its runtime.
- Multi-stage build: compile in one image stage and copy only runtime artifacts into the final image.
- Compose/local orchestration: define multiple containers and their networks/volumes for development.
- Immutable image deployment: build once and deploy the same image across environments.

Real-World Example

A Laravel API is packaged with PHP and required extensions. CI builds a versioned image; Kubernetes later runs multiple replicas from that exact image.

Interview Follow-Up Questions

- What belongs in an image versus a secret/configuration?
- How do you make images smaller and safer?
- What is the difference between a container and a virtual machine?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

8. Kubernetes

Definition

An orchestration platform that schedules containers, manages desired state, provides service discovery, supports rolling updates, and can recover workloads when nodes or containers fail.

Implementation / Usage Types

- Deployment: manages stateless replicas and rolling updates.
- StatefulSet: provides stable identity/storage patterns for stateful workloads.
- Service: gives a stable virtual endpoint for a set of pods.
- DaemonSet: runs a pod on selected nodes, commonly for agents.
- Job/CronJob: runs finite or scheduled workloads.

Real-World Example

A payment API runs as a Deployment with ten replicas. A Service provides stable discovery, readiness probes control traffic, and an HPA scales replicas based on demand.

Interview Follow-Up Questions

- How does Kubernetes decide where a pod runs?
- What is the difference between readiness and liveness?
- How would you safely perform a cluster upgrade?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

9. Service Discovery

Definition

Allows one service to locate another without hard-coding instance addresses. It becomes essential when instances are dynamic because of scaling, deployment, or failure.

Implementation / Usage Types

- DNS-based discovery: service names resolve to current endpoints.
- Registry-based discovery: services register and clients/query systems such as Consul-style registries.
- Client-side discovery: client chooses an instance from discovered endpoints.
- Server-side discovery: a proxy/load balancer selects an instance.

Real-World Example

An order service calls payment-service rather than a fixed IP. Kubernetes DNS resolves the service name to currently available pods.

Interview Follow-Up Questions

- What happens when an endpoint disappears between discovery and connection?
- Where should health checking occur?
- How does discovery interact with load balancing?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

10. Circuit Breakers

Definition

Prevents repeated calls to a failing dependency from consuming resources and causing cascading failure. It usually moves between closed, open, and half-open states.

Implementation / Usage Types

- Closed: calls flow normally while failures are measured.
- Open: calls fail fast for a cooldown period instead of hitting the dependency.
- Half-open: a small number of test calls determine whether recovery occurred.

Real-World Example

If a recommendation service starts timing out, the product API opens its circuit and returns products without recommendations instead of waiting for every request to fail.

Interview Follow-Up Questions

- What failure threshold should open the circuit?
- What fallback is safe for the business?
- How do you avoid a thundering herd when the circuit closes?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

11. Timeouts

Definition

Places an upper bound on how long a request, connection, or operation may wait. Timeouts prevent threads, connections, and queues from being occupied indefinitely.

Implementation / Usage Types

- Connect timeout: limits establishing a connection.
- Read/request timeout: limits waiting for response data.
- End-to-end deadline: gives a whole request a bounded lifetime across services.
- Idle timeout: closes inactive connections after a period.

Real-World Example

A checkout API gives its inventory call a 300 ms deadline. If inventory does not respond, checkout follows a defined fallback rather than waiting several seconds.

Interview Follow-Up Questions

- How should timeout values be selected?
- Why can retries make a timeout problem worse?
- How do you propagate deadlines across services?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

12. Retries

Definition

Attempts a failed operation again when the failure is likely transient. Retries must be bounded and paired with timeouts and backoff.

Implementation / Usage Types

- Immediate retry: useful only for very short transient errors and low contention.
- Fixed-delay retry: waits the same duration between attempts.
- Exponential-backoff retry: increases delay after each failure.
- Conditional retry: retries only selected status codes or error classes.

Real-World Example

A service retries a database connection after a temporary network error, but does not retry a validation error or a non-idempotent payment charge without an idempotency mechanism.

Interview Follow-Up Questions

- Which failures are retryable?
- How many attempts are enough?
- How can retries create a retry storm?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

13. Exponential Backoff

Definition

Gradually increases the delay between retries, reducing pressure on a recovering dependency and spreading repeated attempts over time.

Implementation / Usage Types

- Pure exponential: delay grows as a power of two.
- Exponential with cap: growth stops at a maximum delay.
- Jittered backoff: adds randomness so many clients do not retry simultaneously.
- Decorrelated jitter: varies delays while avoiding synchronized retry patterns.

Real-World Example

After a transient 503, clients wait roughly 100 ms, 200 ms, 400 ms, then a capped interval, with jitter applied before each retry.

Interview Follow-Up Questions

- Why is jitter important?
- How do you combine backoff with a request deadline?
- When should a client stop retrying immediately?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

14. Idempotency

Definition

An operation is idempotent when repeating the same logical request produces the same intended result as performing it once. It is crucial when clients or infrastructure may retry requests.

Implementation / Usage Types

- Naturally idempotent methods: GET, PUT, and DELETE are designed around idempotent semantics.
- Idempotency key: client sends a unique key for a business operation.
- Deduplication record: server stores processed request IDs and results.
- Upsert: create-or-update semantics can reduce duplicate creation.

Real-World Example

A payment client sends an Idempotency-Key with a charge request. If the network fails after the charge succeeds, the retry returns the stored result instead of charging again.

Interview Follow-Up Questions

- Where should the idempotency record live?
- How long should keys be retained?
- What happens if the first request is still processing when a duplicate arrives?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

15. Message Queues

Definition

Decouple producers from consumers by storing messages until consumers can process them. Queues help absorb bursts and allow asynchronous work.

Implementation / Usage Types

- Point-to-point queue: one message is normally processed by one consumer.
- Work queue: multiple workers compete for jobs.
- Priority queue: higher-priority messages are processed first.
- Delayed queue: messages become available after a delay.

Real-World Example

An e-commerce checkout publishes an email job to a queue. Worker processes send the email independently, so checkout does not wait for the email provider.

Interview Follow-Up Questions

- How do you handle duplicate messages?
- What is the difference between at-most-once and at-least-once delivery?
- How do you size consumers during a traffic spike?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

16. Pub/Sub

Definition

Separates publishers from subscribers through topics or channels. A published event can be delivered to multiple independent consumers.

Implementation / Usage Types

- Topic-based pub/sub: producers publish to named topics.
- Fan-out: multiple subscriptions independently consume the same event.
- Consumer groups: instances share the work of a subscription.
- Replayable event streams: consumers can re-read retained events.

Real-World Example

An order-created event is published once. Billing, analytics, notifications, and fraud services each consume it through their own subscription.

Interview Follow-Up Questions

- How do you prevent one slow subscriber from blocking others?
- How is pub/sub different from a queue?
- When do you need replay?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

17. Event-Driven Architecture

Definition

Structures systems around events representing meaningful changes in state. Producers emit events and consumers react asynchronously, reducing direct coupling.

Implementation / Usage Types

- Event notification: event says something happened; consumer fetches details if needed.
- Event-carried state: event contains enough data for consumers to act.
- Event sourcing: state is reconstructed from an ordered event history.
- Streaming architecture: continuous event processing supports analytics and real-time reactions.

Real-World Example

When an order is placed, the order service emits an event. Inventory reserves stock, fraud evaluates risk, and notifications send confirmation independently.

Interview Follow-Up Questions

- Which events deserve durable retention?
- How do you evolve event schemas?
- How do you reason about eventual consistency in the user experience?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

18. Distributed Transactions

Definition

Coordinates changes across multiple resources or services that cannot share one local database transaction. The core challenge is atomicity without a single transaction boundary.

Implementation / Usage Types

- Two-phase commit: coordinator asks participants to prepare and then commit; strong coordination but higher coupling.
- Saga: breaks work into local transactions with compensating actions.
- Transactional outbox: atomically stores a business change and its outgoing event in one database transaction.
- Workflow orchestration: a durable workflow coordinates multi-step operations.

Real-World Example

A loan application updates customer state, creates an application record, and triggers downstream processing. An outbox ensures the event is not lost when the database commit succeeds.

Interview Follow-Up Questions

- When is 2PC justified?
- Why is a Saga not the same as a database rollback?
- How do you make a distributed workflow observable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

19. Saga Pattern

Definition

Models a distributed business transaction as a sequence of local transactions. If a later step fails, compensating actions undo the business effect of earlier steps.

Implementation / Usage Types

- Choreography: services react to each other's events without a central coordinator.
- Orchestration: a workflow coordinator tells each service which step to perform.
- Forward recovery: retry or repair the failed step.
- Compensation: execute a business reversal such as releasing inventory or refunding a payment.

Real-World Example

For an order, inventory is reserved, payment is authorized, and shipment is created. If shipment creation fails, the Saga releases inventory and voids payment authorization.

Interview Follow-Up Questions

- When is orchestration preferable to choreography?
- What if compensation itself fails?
- How do you make Saga state recoverable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

20. Dead Letter Queues

Definition

A holding area for messages that could not be processed successfully after defined attempts or due to non-retryable conditions. It prevents poison messages from blocking normal processing.

Implementation / Usage Types

- Retry-then-DLQ: retry a bounded number of times, then isolate the message.
- Parking queue: hold problematic messages for manual or controlled remediation.
- Quarantine by error type: route known bad payloads separately.
- Replay workflow: fix the cause and re-submit selected messages.

Real-World Example

A billing worker cannot parse a malformed invoice event. After three attempts, the event moves to a DLQ where operators inspect the payload and replay it after correcting the producer.

Interview Follow-Up Questions

- What metadata should a DLQ message contain?
- How do you detect a growing DLQ?
- How do you safely replay messages without duplicate business effects?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

21. Cron Jobs

Definition

Runs scheduled tasks at defined times or intervals. Cron is useful for periodic maintenance, reports, reconciliation, and batch processing.

Implementation / Usage Types

- Fixed schedule: run at a specific time or interval.
- Distributed scheduler: coordinates jobs across multiple nodes.
- Queue-backed schedule: scheduler creates jobs that workers process asynchronously.
- Workflow scheduler: manages dependencies, retries, and execution history.

Real-World Example

A finance platform runs nightly reconciliation. The scheduler creates a reconciliation job, and workers process accounts in parallel while recording checkpoints.

Interview Follow-Up Questions

- How do you prevent duplicate execution?
- What happens if a job runs longer than its interval?
- How do you handle missed schedules during an outage?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

22. WebSockets

Definition

Provides a persistent, bidirectional connection between client and server. Either side can send messages without opening a new HTTP request for each update.

Implementation / Usage Types

- Single WebSocket connection: client maintains one long-lived channel.
- Room/channel model: server routes messages to groups of connected clients.
- Pub/sub-backed WebSockets: multiple gateway instances share events through a broker.
- Presence model: track connection state for online/offline experiences.

Real-World Example

A trading dashboard maintains a WebSocket connection so price updates arrive immediately instead of polling every few seconds.

Interview Follow-Up Questions

- How do you scale WebSockets across multiple servers?
- How do you recover after a client disconnects?
- When is Server-Sent Events a better choice?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

23. Long Polling

Definition

A client sends an HTTP request that the server holds until data is available or a timeout occurs. The client then immediately starts another request.

Implementation / Usage Types

- Fixed long-poll timeout: server waits for a bounded period.
- Event-backed polling: a request waits on an event source.
- Cursor-based long polling: client asks for changes after a known sequence/cursor.
- Backoff on idle: client reduces request frequency when updates are rare.

Real-World Example

A notification service holds a request for up to 30 seconds. When a notification arrives, it responds immediately and the browser opens the next request.

Interview Follow-Up Questions

- What are the scalability costs of long polling?
- How do proxies/load balancers affect long-lived requests?
- Why might WebSockets be preferable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

24. Server-Sent Events

Definition

Uses a long-lived HTTP response to stream server-to-client events. It is one-way: the server pushes updates while the client uses normal HTTP for commands.

Implementation / Usage Types

- Event stream: server sends named or typed events over one HTTP connection.
- Last-event ID: client can tell the server where it stopped after reconnecting.
- Heartbeat: periodic comments/data keep intermediaries from closing idle connections.
- Fan-out through broker: multiple application nodes share event updates.

Real-World Example

A deployment dashboard uses SSE to stream build status from the server to a browser without requiring a bidirectional socket.

Interview Follow-Up Questions

- When would SSE beat WebSockets?
- How do reconnects work?
- What happens when thousands of clients subscribe to the same stream?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

25. Database Indexing

Definition

Creates an additional data structure that helps the database locate rows without scanning the entire table. Indexes improve reads but add storage and write overhead.

Implementation / Usage Types

- B-tree index: general-purpose ordered index for equality and range queries.
- Hash index: optimized for equality in engines that support it.
- Composite index: covers multiple columns; column order matters.
- Unique index: enforces uniqueness while accelerating lookup.
- Full-text/vector index: specialized for text or semantic search.

Real-World Example

An orders table frequently filters by customer_id and created_at. A composite index on (customer_id, created_at) can support that access pattern efficiently.

Interview Follow-Up Questions

- How do you choose index order?
- Why can too many indexes hurt writes?
- How would you verify an index actually helps?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

26. Query Optimization

Definition

Improves database query performance by reducing work such as rows scanned, joins performed, sorting, network transfer, or repeated execution.

Implementation / Usage Types

- Execution-plan optimization: inspect the planner's chosen operations.
- Index optimization: add or adjust indexes for actual predicates and joins.
- Query rewrite: simplify filters, joins, subqueries, or aggregation.
- Data-access optimization: pagination, batching, projection, and precomputation.

Real-World Example

A slow customer report scans millions of rows. The team examines the execution plan, adds a selective index, limits selected columns, and changes pagination to keyset pagination.

Interview Follow-Up Questions

- What does an execution plan tell you?
- How do statistics influence query plans?
- When can an index make a query slower?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

27. N+1 Queries

Definition

A pattern where an initial query fetches N records and then the application performs one additional query per record, producing 1+N database calls.

Implementation / Usage Types

- Eager loading: fetch related data in advance.
- Batch loading: fetch related records in one query using a set of IDs.
- Join-based retrieval: return related data through a database join.
- Data-loader pattern: coalesce requests during one execution cycle.

Real-World Example

A page loads 100 orders with one query, then fetches customer details 100 more times. Batch loading reduces the pattern to a small number of queries.

Interview Follow-Up Questions

- How do you detect N+1 in production?
- When is eager loading too expensive?
- How would you fix N+1 in a GraphQL API?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

28. Connection Pooling

Definition

Maintains reusable database or network connections so applications avoid paying connection-establishment cost for every operation.

Implementation / Usage Types

- Fixed-size pool: maintains a bounded number of connections.
- Dynamic pool: grows and shrinks within configured limits.
- Per-instance pool: each application instance owns its own connections.
- Proxy-managed pool: a connection proxy multiplexes or manages backend connections.

Real-World Example

Twenty application pods each maintain a pool of 20 database connections. Capacity planning must consider the aggregate 400 potential connections, not just the per-pod value.

Interview Follow-Up Questions

- How do you size a database connection pool?
- What happens when the pool is exhausted?
- Why can scaling pods accidentally overload a database?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

29. Read Replicas

Definition

Copies database data to one or more replica nodes that primarily serve read traffic. Replication lag means replicas may temporarily be behind the primary.

Implementation / Usage Types

- Single read replica: offload a portion of read traffic.
- Multiple replicas: distribute read load across several nodes.
- Read/write splitting: writes go to primary; eligible reads go to replicas.
- Consistency-aware routing: recent writes may be read from primary while older reads use replicas.

Real-World Example

A product catalog receives heavy reads. Writes go to the primary database while search/listing reads use replicas, with routing rules for operations that require fresh data.

Interview Follow-Up Questions

- How do you handle replication lag?
- Which reads must go to primary?
- How do replicas affect disaster recovery?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

30. Sharding

Definition

Splits a dataset across independent database nodes using a shard key. Each shard owns a subset of data, allowing scale beyond a single database's capacity.

Implementation / Usage Types

- Hash sharding: hashes a key to distribute data evenly.
- Range sharding: assigns key ranges to shards.
- Directory-based sharding: a mapping service identifies the owning shard.
- Geographic sharding: routes data by region or residency requirement.

Real-World Example

A multi-tenant SaaS system shards customers by tenant ID so one database does not contain all tenant data.

Interview Follow-Up Questions

- How do you choose a shard key?
- What causes hot shards?
- How do cross-shard queries and transactions work?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

31. Partitioning

Definition

Divides a logical table or dataset into smaller partitions while retaining one logical access model. It is often used for manageability and query pruning.

Implementation / Usage Types

- Range partitioning: split by ranges such as date.
- List partitioning: split by discrete values.
- Hash partitioning: distribute rows based on a hash.
- Composite partitioning: combine partitioning strategies.

Real-World Example

A large events table is partitioned by month. Queries restricted to recent dates can scan only relevant partitions, and old partitions can be archived more easily.

Interview Follow-Up Questions

- How is partitioning different from sharding?
- What is partition pruning?
- How do you manage partition growth automatically?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

32. Replication

Definition

Maintains copies of data or state across multiple nodes. Replication can improve availability, read capacity, durability, and geographic resilience, depending on the model.

Implementation / Usage Types

- Synchronous replication: acknowledge after replicas confirm; stronger consistency but higher latency.
- Asynchronous replication: acknowledge before replicas catch up; lower latency but possible lag.
- Leader-follower: one node accepts writes and followers replicate them.
- Multi-leader/active-active: multiple nodes accept writes, requiring conflict handling.

Real-World Example

A database asynchronously replicates from a primary to replicas. A failover process promotes a suitable replica if the primary becomes unavailable.

Interview Follow-Up Questions

- What is the durability trade-off of async replication?
- How do you detect replication lag?
- How are write conflicts handled in multi-leader systems?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

33. Leader Election

Definition

Chooses one node to act as the coordinator or leader for a task so competing nodes do not perform conflicting work simultaneously.

Implementation / Usage Types

- Lease-based election: leader holds a time-limited lease.
- Consensus-based election: algorithms such as Raft use quorum agreement.
- Priority-based election: nodes choose the highest-priority eligible member.
- External coordinator: a coordination service stores leader state.

Real-World Example

Three scheduler instances are running, but only one should trigger a monthly billing job. A leader lease ensures one active scheduler at a time and another can take over after failure.

Interview Follow-Up Questions

- What prevents split brain?
- Why are leases sensitive to clock and network issues?
- What should happen when the leader loses connectivity?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

34. CAP Theorem

Definition

States that during a network partition, a distributed data system cannot simultaneously guarantee both strong consistency and availability for every request. Partition tolerance is generally required in distributed networks.

Implementation / Usage Types

- CP choice: preserve consistency and reject or delay some operations during a partition.
- AP choice: continue serving requests while allowing temporary divergence.
- Consistency models: strong, causal, session, and eventual consistency provide different guarantees.
- Quorum design: read/write quorum choices influence consistency and availability.

Real-World Example

A replicated key-value system must decide what happens when nodes cannot communicate. A CP-oriented design may reject writes without a quorum, while an AP design may accept them and reconcile later.

Interview Follow-Up Questions

- What exactly does CAP mean by availability?
- Why is partition tolerance not optional in real distributed networks?
- How does quorum size affect consistency?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

35. Eventual Consistency

Definition

Means replicas may temporarily disagree but converge to the same value if updates stop and communication succeeds. It trades immediate consistency for availability, latency, or scale.

Implementation / Usage Types

- Read-your-writes: a user sees their own latest changes.
- Monotonic reads: a client does not move backward to older data.
- Causal consistency: causally related operations are observed in a valid order.
- Conflict-free replication: structured data types can merge concurrent updates automatically.

Real-World Example

A social feed may tolerate a short delay before a new post appears on every replica, while the author's own view is routed to a source that guarantees read-your-writes.

Interview Follow-Up Questions

- Which user actions cannot tolerate stale data?
- How do you communicate eventual consistency to product teams?
- How do replicas converge after concurrent writes?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

36. Optimistic Locking

Definition

Allows concurrent work to proceed and detects conflicting updates at commit time, commonly using a version number or timestamp.

Implementation / Usage Types

- Version column: update only if the stored version matches the version read.
- ETag/If-Match: HTTP clients provide the version they observed.
- Compare-and-swap: update succeeds only when a value matches.
- Application conflict handling: return a conflict for the caller to merge or retry.

Real-World Example

Two agents edit the same customer record. The second update includes version 17; if the database is already at version 18, the update fails and the UI asks the user to refresh.

Interview Follow-Up Questions

- When is optimistic locking preferable?
- How should the API represent a conflict?
- What happens if conflicts are frequent?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

37. Pessimistic Locking

Definition

Locks data before or during a transaction so competing operations wait or fail instead of updating concurrently.

Implementation / Usage Types

- Row lock: lock selected rows during a transaction.
- Table lock: lock a larger data set when row-level locking is insufficient.
- Advisory lock: application-defined lock managed by the database.
- SELECT FOR UPDATE: commonly used to lock rows that will be changed.

Real-World Example

An inventory reservation transaction locks a stock row before decrementing available quantity so two concurrent orders cannot both reserve the last unit.

Interview Follow-Up Questions

- How do locks affect throughput?
- What causes lock contention?
- How do you prevent deadlocks when using multiple locks?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

38. Distributed Locks

Definition

Coordinates mutually exclusive work across multiple processes or machines. A correct implementation needs ownership, expiry/recovery, and safeguards against stale lock holders.

Implementation / Usage Types

- Lease lock: ownership expires unless renewed.
- Database lock: use transactional or advisory locking in a database.
- Redis-style lock: distributed key with ownership token and expiry.
- Consensus-backed lock: coordination systems provide stronger semantics.

Real-World Example

Only one instance should generate a daily settlement file. A lease prevents duplicate generation, and the job includes a fencing token so a stale worker cannot overwrite newer work.

Interview Follow-Up Questions

- Why is a TTL alone not enough?
- What is a fencing token?
- How would you test a lock under network partitions?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

39. Race Conditions

Definition

Occurs when the outcome depends on the timing or interleaving of concurrent operations. Race conditions can cause lost updates, duplicate work, or inconsistent state.

Implementation / Usage Types

- Check-then-act race: state changes between validation and action.
- Read-modify-write race: concurrent updates overwrite each other.
- Double-submit race: two requests perform the same business action.
- Initialization race: consumers observe partially initialized state.

Real-World Example

Two checkout requests both read stock=1 and both attempt to reserve it. A transaction with a suitable lock or atomic update prevents the double reservation.

Interview Follow-Up Questions

- How do you reproduce a race condition?
- Which database primitives can eliminate it?
- When can idempotency solve the problem instead of locking?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

40. Deadlocks

Definition

Occurs when two or more operations wait indefinitely for resources held by one another. Databases often detect deadlocks and abort one transaction.

Implementation / Usage Types

- Database deadlock: transactions lock rows in incompatible order.
- Application deadlock: threads wait on each other's locks.
- Distributed deadlock: dependencies across services create a cycle.
- Prevention: consistent lock ordering, shorter transactions, and bounded waits.

Real-World Example

Transaction A locks customer then order; transaction B locks order then customer. A consistent ordering rule makes both acquire customer before order and removes the cycle.

Interview Follow-Up Questions

- How does a database detect a deadlock?
- What is the difference between deadlock and lock contention?
- How should an application react after a deadlock victim is aborted?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

41. Memory Leaks

Definition

A memory leak occurs when a process retains objects or resources that are no longer needed, causing memory usage to grow and potentially triggering slowdowns or crashes.

Implementation / Usage Types

- Unreleased references: objects remain reachable accidentally.
- Cache leak: unbounded cache growth.
- Listener/subscription leak: callbacks remain registered.
- Native/resource leak: file descriptors, sockets, or unmanaged memory are not released.

Real-World Example

A Node.js service creates a timer and event listener per request but never removes them. Heap usage grows over hours until the container is restarted.

Interview Follow-Up Questions

- How do you distinguish a leak from legitimate growth?
- Which metrics and heap tools would you use?
- Why can a restart hide rather than fix the root cause?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

42. Garbage Collection

Definition

Automatically reclaims memory that is no longer reachable by the application. GC reduces manual memory-management errors but can consume CPU and introduce pauses depending on runtime and collector.

Implementation / Usage Types

- Tracing collectors: identify reachable objects and reclaim unreachable ones.
- Generational GC: treats young and old objects differently based on allocation behavior.
- Concurrent GC: performs more work alongside application threads.
- Reference counting: tracks references, though cycles require additional handling in many systems.

Real-World Example

A Java service with many short-lived request objects benefits from generational collection, while heap sizing and GC pause metrics remain part of production tuning.

Interview Follow-Up Questions

- What creates GC pressure?
- How can excessive allocations affect latency?
- What signals tell you GC tuning is actually needed?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

43. Thread Safety

Definition

Code is thread-safe when concurrent execution does not produce incorrect results or violate invariants. Shared mutable state is a common source of problems.

Implementation / Usage Types

- Immutable state: avoid mutation so concurrent readers are safe.
- Mutex/lock: serialize access to critical sections.
- Atomic operations: update shared values without a larger lock.
- Thread-local state: give each thread independent mutable state.

Real-World Example

A Java service increments a shared counter from many request threads. Using an atomic counter or appropriate synchronization prevents lost increments.

Interview Follow-Up Questions

- What is a critical section?
- When can locking reduce throughput?
- How is thread safety different from process safety?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

44. Backpressure

Definition

A mechanism by which a fast producer is slowed, bounded, or otherwise controlled when a downstream consumer cannot keep up.

Implementation / Usage Types

- Bounded queue: limits buffered work and eventually rejects or blocks producers.
- Rate control: limit producer throughput.
- Flow control: consumer signals how much data it can accept.
- Load shedding: deliberately reject lower-priority work when capacity is exhausted.

Real-World Example

A video-processing pipeline receives 1,000 jobs/minute while workers can process 600. A bounded queue plus admission control prevents unbounded memory growth.

Interview Follow-Up Questions

- What should happen when the queue is full?
- How does backpressure differ from rate limiting?
- Which requests should be shed first during overload?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

45. Autoscaling

Definition

Automatically adjusts compute capacity according to demand or resource signals. Good autoscaling needs a useful metric, safe bounds, and enough startup capacity.

Implementation / Usage Types

- CPU-based scaling: increase instances when CPU utilization rises.
- Memory-based scaling: respond to memory pressure.
- Request/queue-based scaling: scale on traffic or pending work.
- Scheduled scaling: pre-scale for predictable peaks.

Real-World Example

A ticketing system scales worker pods based on queue depth. During a sale, backlog rises and more workers are added; after the backlog clears, capacity reduces.

Interview Follow-Up Questions

- What metric best predicts user impact?
- How do you prevent oscillation?
- How do you account for cold-start time?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

46. Horizontal Scaling

Definition

Increases capacity by adding more instances. It is the common scaling model for stateless web services and many distributed workloads.

Implementation / Usage Types

- Stateless replicas: add identical application instances.
- Partitioned workers: add consumers that share work.
- Distributed cache/data tier: scale storage or throughput across nodes.
- Autoscaled fleet: automatically add/remove instances.

Real-World Example

A REST API scales from 10 to 50 pods during peak traffic. Sessions are stored outside the pod so requests can be served by any instance.

Interview Follow-Up Questions

- What application state prevents horizontal scaling?
- How does load balancing distribute the new capacity?
- What downstream system becomes the bottleneck first?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

47. Vertical Scaling

Definition

Increases the resources of an existing machine or instance, such as CPU, memory, or I/O capacity.

Implementation / Usage Types

- CPU scaling: use a larger compute class.
- Memory scaling: increase RAM for larger working sets.
- Storage/I/O scaling: increase disk throughput or IOPS.
- Database vertical scaling: move to a larger database instance.

Real-World Example

A database is hitting memory pressure but has moderate traffic. Moving to a larger instance may solve the immediate issue while the team evaluates longer-term partitioning or query improvements.

Interview Follow-Up Questions

- What are the limits of vertical scaling?
- When is vertical scaling cheaper?
- How would you decide between scale-up and scale-out?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

48. CDN

Definition

A content delivery network distributes content through geographically distributed edge locations, reducing latency and origin load for cacheable traffic.

Implementation / Usage Types

- Static asset delivery: images, CSS, JavaScript, and media.
- Dynamic acceleration: optimize network paths even when content is not cached.
- Edge security: combine CDN delivery with WAF and DDoS controls.
- Origin shielding: reduce the number of requests reaching the origin.

Real-World Example

A global retail site serves product images from CDN edge locations. A user in Singapore does not need to fetch every image from a Mumbai origin.

Interview Follow-Up Questions

- What content should be cacheable?
- How do cache headers affect CDN behavior?
- What happens when an origin becomes unavailable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

49. Edge Caching

Definition

Stores content at edge locations close to users. It is a specific caching strategy often implemented through a CDN or edge platform.

Implementation / Usage Types

- TTL-based edge cache: content remains cached for a configured duration.
- Cache-key variation: vary by URL, query, header, or cookie where necessary.
- Stale-while-revalidate: serve stale content while refreshing in the background.
- Edge-compute caching: execute lightweight logic close to the user.

Real-World Example

A public product page is cached at edge nodes for 30 seconds while personalized account pages bypass the shared cache.

Interview Follow-Up Questions

- How do you prevent personalized data from leaking through shared cache?
- What is cache-key design?
- When is stale content acceptable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

50. Cache Invalidation

Definition

Defines how cached data becomes invalid when its source changes. It is one of the hardest cache problems because stale copies may exist in many places.

Implementation / Usage Types

- TTL expiration: let entries become stale after a fixed time.
- Explicit invalidation: delete or update affected keys after a write.
- Versioned keys: change the key when the underlying version changes.
- Write-through/write-behind: coordinate cache updates with writes.

Real-World Example

When a product price changes, the product service invalidates the corresponding cache key. For bulk catalog changes, versioned namespaces make invalidation manageable.

Interview Follow-Up Questions

- What if invalidation fails after a successful database write?
- How do you handle distributed cache invalidation?
- When is short TTL safer than complex invalidation?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

51. Feature Flags

Definition

Separates code deployment from feature activation. A feature can be enabled for selected users, regions, or percentages without redeploying code.

Implementation / Usage Types

- Boolean flag: globally on/off.
- Targeted flag: enable for selected users, tenants, or cohorts.
- Percentage rollout: expose a feature to a controlled fraction of traffic.
- Kill switch: rapidly disable a problematic feature.

Real-World Example

A new checkout flow is deployed dark. Product enables it for internal users, then 5%, 25%, and 100% of traffic while monitoring conversion and errors.

Interview Follow-Up Questions

- How do you prevent flag sprawl?
- How should flag evaluation be made highly available?
- What is the rollback plan if code and old behavior diverge?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

52. Blue-Green Deployment

Definition

Maintains two production environments: one serving traffic and one holding the candidate release. Traffic switches between them after validation.

Implementation / Usage Types

- Full switch: move nearly all traffic at once.
- Weighted switch: move traffic gradually even within a blue-green model.
- Database-compatible deployment: ensure both versions can safely operate during transition.
- Instant rollback: redirect traffic to the previous environment if the new version fails.

Real-World Example

A payments service runs v1 in blue and v2 in green. After smoke and synthetic tests pass, traffic moves to green. The previous environment remains available for rollback.

Interview Follow-Up Questions

- How do you handle database schema changes?
- What capacity cost does blue-green introduce?
- What signals must pass before switching traffic?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

53. Canary Releases

Definition

Releases a new version to a small subset of production traffic first, using real traffic to validate behavior before wider rollout.

Implementation / Usage Types

- Percentage canary: 1%, 5%, 25%, then more.
- User cohort canary: selected tenants or internal users.
- Region canary: release in one geographic region first.
- Metric-driven promotion: expand only when error, latency, and business metrics remain healthy.

Real-World Example

A new recommendation algorithm is exposed to 1% of users. The team compares latency, click-through rate, and error rate before increasing exposure.

Interview Follow-Up Questions

- How do you select canary traffic?
- Which metrics should block promotion?
- How do you avoid false confidence from a small sample?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

54. Rolling Deployments

Definition

Replaces old instances with new ones incrementally rather than replacing the whole fleet at once. It reduces disruption but requires compatibility between versions.

Implementation / Usage Types

- Fixed batch: replace a fixed number or percentage at a time.
- Surge: temporarily run extra capacity while adding new instances.
- Readiness-gated rollout: route traffic only to healthy new instances.
- Partitioned rollout: update groups or zones sequentially.

Real-World Example

A Kubernetes Deployment updates 10 pods in batches, waiting for readiness before replacing more old pods.

Interview Follow-Up Questions

- What happens if the new version is unhealthy halfway through?
- How do you preserve capacity during rollout?
- What compatibility is required between old and new versions?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

55. Rollbacks

Definition

Returns a deployment to a previously known-good version after a release causes unacceptable behavior. Application rollback and database rollback are different problems.

Implementation / Usage Types

- Artifact rollback: redeploy a prior immutable application image.
- Traffic rollback: shift traffic to the old environment.
- Configuration rollback: restore a known-good configuration.
- Forward fix: deploy a corrective release when database changes make code rollback unsafe.

Real-World Example

A release causes elevated 5xx errors. The deployment controller restores the previous application image while the team investigates the defect.

Interview Follow-Up Questions

- When is rollback safer than forward fix?
- How do you roll back a schema change?
- What telemetry should automatically trigger rollback consideration?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

56. Health Checks

Definition

Tests whether a component is functioning sufficiently for its intended role. Health checks guide traffic routing, orchestration, and operational decisions.

Implementation / Usage Types

- Shallow health check: verifies the process is responding.
- Dependency health check: verifies critical dependencies.
- Functional health check: performs a representative business operation.
- Load-balancer health check: determines whether an instance should receive traffic.

Real-World Example

A load balancer checks a lightweight /health endpoint. A deeper diagnostic endpoint checks dependencies but is not used as the traffic gate because one optional dependency should not remove the whole instance.

Interview Follow-Up Questions

- What should a health endpoint avoid doing?
- How can health checks create an outage themselves?
- Which dependencies are critical versus optional?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

57. Liveness & Readiness Probes

Definition

Liveness asks whether a process should be restarted; readiness asks whether it should receive traffic. Confusing the two can turn a dependency outage into a restart storm.

Implementation / Usage Types

- Liveness probe: detects a stuck or irrecoverable process.
- Readiness probe: controls whether the instance enters service.
- Startup probe: gives slow-starting applications time before liveness checks begin.
- Dependency-aware readiness: temporarily removes an instance when a required dependency is unavailable.

Real-World Example

A Kubernetes pod takes 40 seconds to start. A startup probe prevents premature restarts, while readiness stays false until the application can safely serve traffic.

Interview Follow-Up Questions

- Why should liveness usually be shallow?
- What happens when readiness fails?
- When is a startup probe necessary?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

58. Monitoring

Definition

Collects and evaluates operational signals so teams can understand system health, capacity, and user impact. Monitoring should connect technical signals to actionable decisions.

Implementation / Usage Types

- Infrastructure monitoring: CPU, memory, disk, network.
- Application monitoring: errors, request rates, latency, saturation.
- Business monitoring: payments, orders, conversion, failed transactions.
- Synthetic monitoring: periodically simulate important user journeys.

Real-World Example

A payment platform monitors API latency, database saturation, payment success rate, and checkout completion. A spike in failures triggers an incident workflow.

Interview Follow-Up Questions

- Which metrics should page someone?
- How do you avoid alert fatigue?
- How do technical and business monitoring complement each other?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

59. Logging

Definition

Records structured or textual events produced by applications and infrastructure. Good logs support diagnosis without becoming a security or cost problem.

Implementation / Usage Types

- Structured logging: emit machine-readable fields such as request_id and status.
- Centralized logging: aggregate logs for search and correlation.
- Audit logging: record security or business actions with stronger retention controls.
- Sampling: reduce volume for repetitive or low-value events.

Real-World Example

A service logs request ID, endpoint, status, latency, and safe business identifiers as structured JSON, allowing operators to correlate an error across services.

Interview Follow-Up Questions

- What should never be logged?
- How do you correlate logs across services?
- How do you control logging cost?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

60. Distributed Tracing

Definition

Tracks a request as it crosses multiple services, representing work as spans connected by trace context. It helps identify where latency and errors originate.

Implementation / Usage Types

- Trace/span model: one trace contains many timed spans.
- Context propagation: carry trace IDs through HTTP, messaging, and RPC.
- Sampling: trace a percentage or selected requests.
- Tail-based sampling: decide what to retain after seeing the completed trace.

Real-World Example

A checkout request spans API gateway, cart, inventory, payment, and notification services. Tracing shows that inventory consumes most of the request's latency budget.

Interview Follow-Up Questions

- How is tracing different from logging?
- How do you propagate trace context through queues?
- When should you use tail-based sampling?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

61. Metrics

Definition

Numeric measurements tracked over time. Metrics are efficient for aggregation, alerting, capacity planning, and service-level objectives.

Implementation / Usage Types

- Counter: monotonically increasing count such as requests.
- Gauge: current value such as queue depth.
- Histogram: distribution of values such as request latency.
- Summary: precomputed quantile-style statistics in some monitoring systems.

Real-World Example

A service exports request count, error count, latency histogram, active connections, and queue depth. Dashboards show both technical and customer-impacting trends.

Interview Follow-Up Questions

- Why are histograms useful for latency?
- What metric labels can create high cardinality?
- Which metrics are best for SLOs?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

62. Alerts

Definition

Notifications triggered when measured conditions indicate a problem requiring action. Effective alerts are actionable, bounded, and tied to user impact.

Implementation / Usage Types

- Threshold alert: trigger above or below a fixed value.
- Rate alert: detect a change in errors or requests over time.
- Burn-rate alert: detect rapid consumption of an SLO error budget.
- Composite alert: combine several signals to reduce noise.

Real-World Example

Instead of paging on CPU > 80%, a service pages when error rate is elevated and the SLO burn rate indicates customers are being affected.

Interview Follow-Up Questions

- What makes an alert actionable?
- Which alerts belong in tickets rather than pages?
- How do you tune noisy alerts?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

63. SLOs

Definition

Service Level Objectives define target reliability or performance for a service over a measurement period. They turn reliability into an explicit engineering and product decision.

Implementation / Usage Types

- Availability SLO: target percentage of successful requests.
- Latency SLO: target percentage of requests under a latency threshold.
- Freshness SLO: target percentage of data meeting a freshness bound.
- Correctness SLO: target percentage of operations producing valid outcomes.

Real-World Example

A checkout API may target 99.9% successful requests per month and a latency objective for 99% of valid requests. These targets guide alerting and release decisions.

Interview Follow-Up Questions

- How is an SLO different from an SLA?
- How should SLOs influence roadmap trade-offs?
- What happens when the error budget is exhausted?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

64. SLIs

Definition

Service Level Indicators are measurements used to evaluate whether a service is meeting an SLO. An SLI should represent a user-relevant aspect of service quality.

Implementation / Usage Types

- Availability SLI: successful requests divided by valid requests.
- Latency SLI: requests meeting a defined latency threshold.
- Durability SLI: data preserved without loss over a period.
- Freshness SLI: data updates delivered within a target window.

Real-World Example

For a payment API, the availability SLI can measure successful eligible transactions, excluding clearly invalid client requests from the reliability numerator.

Interview Follow-Up Questions

- What should be included in the SLI denominator?
- Why can an SLI be technically correct but not user-relevant?
- How do you instrument an SLI?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

65. Error Budgets

Definition

Represents the amount of unreliability allowed by an SLO. It gives teams a quantitative way to balance feature delivery with reliability work.

Implementation / Usage Types

- Time budget: allowable downtime derived from an availability SLO.
- Request budget: allowable failed requests over a measurement window.
- Burn rate: speed at which the budget is being consumed.
- Policy budget: organizational rules for what happens as the budget is depleted.

Real-World Example

A 99.9% monthly availability target allows roughly 43.2 minutes of unavailability in a 30-day month. If a release rapidly consumes the budget, the team may pause risky changes.

Interview Follow-Up Questions

- Why is error budget better than a vague reliability goal?
- How do you calculate burn rate?
- What should happen when the budget reaches zero?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

66. Observability

Definition

Describes how well a system's internal state can be inferred from its external outputs, typically using logs, metrics, and traces plus contextual signals.

Implementation / Usage Types

- Metrics: reveal trends and aggregate behavior.
- Logs: provide detailed event context.
- Traces: show request paths and timing across components.
- Profiling: reveals CPU, memory, and runtime behavior.

Real-World Example

During a checkout incident, metrics show elevated latency, traces identify a slow inventory call, and logs reveal a connection-pool exhaustion error.

Interview Follow-Up Questions

- How is observability different from monitoring?
- What questions should an observability platform answer?
- How do you control telemetry cost?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

67. Secrets Management

Definition

Protects sensitive values such as API keys, passwords, certificates, and tokens throughout their lifecycle. Secrets should not be embedded in source code or images.

Implementation / Usage Types

- Environment injection: supply secrets at runtime.
- Secret manager: centrally store and retrieve encrypted secrets.
- Dynamic secrets: generate short-lived credentials.
- Rotation: replace secrets periodically or after suspected compromise.

Real-World Example

A service retrieves a database credential from a managed secret store at startup rather than storing the password in Git or a Docker image.

Interview Follow-Up Questions

- How do you rotate a credential without downtime?
- Who should be allowed to read each secret?
- How do you detect secrets accidentally committed to Git?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

68. IAM

Definition

Identity and Access Management controls who or what can access resources and what actions they are allowed to perform.

Implementation / Usage Types

- RBAC: permissions are assigned through roles.
- ABAC: access depends on attributes such as user, resource, or environment.
- Service identity: workloads receive identities instead of shared static keys.
- Least privilege: grant only the permissions required for the task.

Real-World Example

A deployment pipeline assumes a deployment role that can update only the production service it owns, while developers have read-only production access.

Interview Follow-Up Questions

- How do you implement least privilege?
- What is the difference between authentication and authorization?
- How do you audit privileged access?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

69. OAuth

Definition

An authorization framework that lets a client obtain limited access to a resource on behalf of a user or itself without sharing the user's password with the client.

Implementation / Usage Types

- Authorization Code + PKCE: common for browser/mobile public clients.
- Client Credentials: service-to-service authorization without a user.
- Refresh token flow: obtain new access tokens without repeating the user authorization step.
- Scoped access: tokens carry limited permissions.

Real-World Example

A banking mobile app uses Authorization Code with PKCE to obtain scoped access to account data without giving the mobile app the user's password.

Interview Follow-Up Questions

- Why is PKCE important?
- What is the difference between OAuth and authentication?
- How should refresh tokens be protected?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

70. JWT Rotation

Definition

Replaces signing or access-token credentials periodically and supports invalidation or key rollover without breaking all active clients.

Implementation / Usage Types

- Signing-key rotation: introduce a new key while accepting the previous key during a transition.
- Access-token rotation: issue short-lived tokens.
- Refresh-token rotation: replace refresh tokens after use and detect reuse.
- Key-set publishing: expose active public verification keys through a JWKS-style endpoint.

Real-World Example

A service begins signing new JWTs with key K2 while still accepting K1 for tokens issued before the rotation window. After expiry, K1 is retired.

Interview Follow-Up Questions

- How do you rotate keys without downtime?
- How do you detect refresh-token replay?
- What should token TTLs be based on?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

71. TLS

Definition

Transport Layer Security protects network communication by authenticating endpoints and encrypting data in transit. HTTPS is HTTP carried over TLS.

Implementation / Usage Types

- One-way server authentication: client validates the server certificate.
- Mutual TLS: both sides authenticate using certificates.
- Certificate automation: renew certificates before expiration.
- TLS termination: decrypt at a trusted proxy or gateway.

Real-World Example

A public API uses HTTPS with an automatically renewed certificate. Internal high-trust service traffic may additionally use mutual TLS for workload identity.

Interview Follow-Up Questions

- What does a certificate prove?
- Where should TLS terminate in a microservice architecture?
- What is the operational risk of certificate expiration?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

72. Encryption at Rest

Definition

Protects stored data when it is not actively moving through the network. It reduces exposure if storage media, snapshots, or backups are accessed improperly.

Implementation / Usage Types

- Volume/storage encryption: encrypt disks or object storage.
- Database encryption: encrypt database files or tablespaces.
- Application-level encryption: encrypt selected sensitive fields before storage.
- Backup encryption: ensure snapshots and backup copies are encrypted too.

Real-World Example

A customer database uses managed disk encryption, while highly sensitive fields are additionally encrypted by the application using a managed key service.

Interview Follow-Up Questions

- What does storage encryption not protect against?
- When is application-level encryption necessary?
- How should encryption keys be rotated?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

73. Encryption in Transit

Definition

Encrypts data while it travels between clients, services, proxies, or databases. TLS is the common mechanism for IP-based application traffic.

Implementation / Usage Types

- HTTPS: TLS-protected HTTP.
- mTLS: mutual certificate authentication plus encryption.
- VPN/private links: protect network paths but do not necessarily replace application-level TLS.
- Database TLS: encrypt application-to-database traffic.

Real-World Example

A payment service uses HTTPS externally and TLS for database connections so credentials and customer data are not transmitted as plaintext.

Interview Follow-Up Questions

- Is a private network enough without TLS?
- How do you validate certificate trust?
- What is the performance impact of TLS and how is it managed?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

74. WAF

Definition

A Web Application Firewall inspects HTTP traffic and applies rules to block or challenge common web attacks and unwanted request patterns.

Implementation / Usage Types

- Managed rules: provider-maintained attack signatures.
- Custom rules: organization-specific conditions.
- Rate-based rules: limit abusive request patterns.
- Virtual patching: block exploitation attempts before application code is changed.

Real-World Example

A public checkout endpoint uses WAF rules to block obvious SQL injection and malicious payloads while rate-based controls reduce abusive automated traffic.

Interview Follow-Up Questions

- What attacks can a WAF detect poorly?
- How do false positives affect production?
- Why is a WAF not a substitute for secure application code?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

75. DDoS Protection

Definition

Defends services against distributed denial-of-service attacks that attempt to exhaust network, protocol, or application resources.

Implementation / Usage Types

- Network-layer mitigation: absorb volumetric traffic upstream.
- Transport-layer mitigation: filter protocol abuse.
- Application-layer mitigation: identify expensive or malicious HTTP behavior.
- CDN/edge absorption: distribute and filter traffic before it reaches origin.

Real-World Example

A public API is placed behind a DDoS-protection service and CDN. Volumetric traffic is absorbed at the edge while WAF rules handle application-layer patterns.

Interview Follow-Up Questions

- How do you distinguish a DDoS attack from a legitimate traffic spike?
- Which protections should exist before traffic reaches the origin?
- How do you test resilience safely?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

76. CORS

Definition

Cross-Origin Resource Sharing is a browser security mechanism that controls whether JavaScript from one origin may access resources on another origin.

Implementation / Usage Types

- Allowlist origins: explicitly permit known web origins.
- Simple requests: may be sent without a preflight depending on method and headers.
- Preflight: browser sends OPTIONS to ask whether a cross-origin request is allowed.
- Credentialed CORS: cookies/credentials require stricter origin handling.

Real-World Example

A React app hosted on app.example.com calls api.example.com. The API returns CORS headers allowing only the intended application origin.

Interview Follow-Up Questions

- Why is '*' problematic with credentials?
- What triggers a preflight?
- Does CORS protect server-to-server requests?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

77. CSRF

Definition

Cross-Site Request Forgery tricks a user's browser into sending an unwanted authenticated request to a site. It primarily affects browser-based authentication patterns such as cookies.

Implementation / Usage Types

- CSRF token: server issues an unpredictable token that must accompany state-changing requests.
- SameSite cookies: restrict when cookies are sent cross-site.
- Origin/Referer validation: verify request source where appropriate.
- Double-submit cookie: token is placed in both cookie and request data.

Real-World Example

A banking web app uses SameSite cookies plus CSRF tokens on money-transfer forms, reducing the chance that another site can cause a transfer using the victim's browser session.

Interview Follow-Up Questions

- Why are bearer tokens in Authorization headers different from cookies for CSRF?
- Which requests require CSRF protection?
- How do you test CSRF defenses?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

78. SQL Injection

Definition

Occurs when untrusted input changes the meaning of a SQL statement. The primary defense is parameterized queries or safe query builders, not input filtering alone.

Implementation / Usage Types

- Parameterized queries: values are sent separately from SQL structure.
- Prepared statements: database/client compiles a statement with bound values.
- ORM/query builder: generates parameterized SQL when used correctly.
- Least-privilege DB account: limits damage if injection occurs.

Real-World Example

A login endpoint originally concatenates email input into SQL. Replacing string concatenation with bound parameters prevents the input from becoming SQL syntax.

Interview Follow-Up Questions

- Why does escaping alone provide weaker guarantees?
- How do you find injection vulnerabilities?
- What database permissions should an application account have?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

79. XSS

Definition

Cross-Site Scripting occurs when attacker-controlled content executes as script in a victim's browser. The main defenses are context-aware output encoding, safe templating, and strong content security policy.

Implementation / Usage Types

- Stored XSS: malicious content is persisted and shown to users.
- Reflected XSS: payload is reflected from a request into a response.
- DOM-based XSS: client-side code turns unsafe data into executable DOM content.
- Defense-in-depth: encoding, sanitization, CSP, and secure cookie flags.

Real-World Example

A comment field stores user text. The frontend renders it as text rather than raw HTML, preventing a script payload from executing in another user's browser.

Interview Follow-Up Questions

- Why is HTML sanitization context-dependent?
- How does CSP reduce XSS impact?
- What is the difference between stored and reflected XSS?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

80. SSRF

Definition

Server-Side Request Forgery occurs when an attacker can influence a server into making requests to unintended destinations, potentially reaching internal services or cloud metadata endpoints.

Implementation / Usage Types

- Allowlist destinations: permit only known external hosts.
- URL validation: parse and validate scheme, host, port, and resolved address.
- Network isolation: restrict outbound access from application workloads.
- Metadata protection: use cloud controls that prevent unauthenticated metadata access.

Real-World Example

An image-import service accepts a URL. It validates approved domains and blocks private/link-local address ranges so attackers cannot make the server call internal administrative endpoints.

Interview Follow-Up Questions

- Why is hostname validation alone insufficient?
- How can DNS rebinding complicate defenses?
- What network controls provide defense in depth?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

81. Database Migrations

Definition

Controlled changes to database structure or data that are versioned and repeatable across environments.

Implementation / Usage Types

- Schema migration: add/change/drop tables, columns, indexes, or constraints.
- Data migration: transform existing records.
- Expand-and-contract: add new structure, migrate traffic/data, then remove old structure.
- Online migration: change large tables with minimal locking or downtime.

Real-World Example

A service needs to rename a customer field. It first adds the new column, writes both fields, backfills old records, switches reads, then removes the old column after all clients are compatible.

Interview Follow-Up Questions

- Why is expand-and-contract safer?
- How do you migrate a billion-row table?
- How do you roll back a migration safely?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

82. Schema Versioning

Definition

Tracks the evolution of a data schema so producers and consumers can coordinate changes without ambiguity.

Implementation / Usage Types

- Database schema versioning: migration files identify database versions.
- API schema versioning: contracts distinguish compatible/incompatible API shapes.
- Event schema versioning: consumers support multiple event versions during transition.
- Backward-compatible evolution: add optional fields before removing old ones.

Real-World Example

An order event adds an optional currency field. Existing consumers ignore unknown fields, while newer consumers use it. The old field is removed only after consumers have migrated.

Interview Follow-Up Questions

- What makes a schema change backward compatible?
- How do you handle old consumers?
- How do you govern event contracts across teams?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

83. Disaster Recovery

Definition

Plans and capabilities for restoring critical services after severe failures such as regional outages, data corruption, or infrastructure loss.

Implementation / Usage Types

- Backup/restore: recover from stored copies.
- Warm standby: maintain a partially ready secondary environment.
- Hot standby: keep a near-ready replica environment.
- Active-active: operate multiple regions simultaneously.

Real-World Example

A financial service maintains a secondary region with replicated data and tested infrastructure. During a regional disaster, traffic is redirected after defined recovery checks.

Interview Follow-Up Questions

- What are RTO and RPO?
- Which dependencies must be included in the DR plan?
- How often should DR recovery be tested?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

84. Backups

Definition

Creates recoverable copies of data so systems can be restored after accidental deletion, corruption, ransomware, or infrastructure failure.

Implementation / Usage Types

- Full backup: captures the complete dataset.
- Incremental backup: stores changes since a previous backup.
- Snapshot: point-in-time storage representation.
- Continuous/PITR: retain logs or changes so data can be restored to a selected time.

Real-World Example

A production database keeps encrypted backups and point-in-time recovery. A quarterly restore test verifies that backups are actually usable.

Interview Follow-Up Questions

- What is the difference between backup and replication?
- How do you validate backup integrity?
- How long should backups be retained?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

85. Failover

Definition

Moves service responsibility from a failed or degraded component to a healthy alternative.

Implementation / Usage Types

- Automatic failover: monitoring detects failure and triggers promotion/routing.
- Manual failover: operators initiate the switch for controlled incidents.
- Database failover: promote a replica or secondary cluster.
- Traffic failover: DNS, load balancer, or global routing shifts clients.

Real-World Example

When a primary database becomes unavailable, an eligible replica is promoted and application connections are redirected after health and consistency checks.

Interview Follow-Up Questions

- What prevents unnecessary failovers?
- How do clients reconnect after failover?
- What data could be lost with asynchronous replication?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

86. Multi-Region Deployment

Definition

Runs application capacity in more than one geographic region to improve resilience, latency, or data-locality options.

Implementation / Usage Types

- Active-passive: one region serves while another waits.
- Active-active: multiple regions serve traffic concurrently.
- Regional sharding: each region owns a subset of users/data.
- Global traffic routing: direct users to healthy or optimal regions.

Real-World Example

A global SaaS product serves users from multiple regions. Stateless services run in each region, while data ownership and replication rules define where writes occur.

Interview Follow-Up Questions

- What is the consistency model across regions?
- How do you handle cross-region writes?
- How do you test regional failure without causing a customer outage?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

87. Chaos Engineering

Definition

Deliberately introduces controlled failures to validate assumptions about resilience and recovery.

Implementation / Usage Types

- Instance failure: terminate or isolate selected workloads.
- Network fault: add latency, packet loss, or partition behavior.
- Dependency failure: simulate a database or third-party outage.
- Capacity experiment: test behavior near resource limits.

Real-World Example

A team injects latency into a non-production payment dependency and verifies that timeouts, circuit breakers, fallbacks, and alerts behave as designed before repeating a controlled production experiment.

Interview Follow-Up Questions

- What is the blast-radius limit?
- Which hypothesis are you testing?
- How do you prove the system recovered correctly?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

88. Cost Optimization

Definition

Reduces infrastructure and operating cost while preserving required reliability, performance, and business outcomes.

Implementation / Usage Types

- Rightsizing: match resource capacity to actual demand.
- Autoscaling: reduce idle capacity.
- Storage lifecycle: move old data to cheaper tiers.
- Architecture optimization: reduce unnecessary network, compute, or database work.
- Reserved/committed capacity: lower cost for predictable workloads.

Real-World Example

A service discovers that non-production environments run 24/7 despite low usage. Scheduled shutdowns and smaller instances reduce cost without affecting production.

Interview Follow-Up Questions

- What costs are fixed versus variable?
- How do you optimize without damaging reliability?
- How do you measure savings after a change?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

89. Cold Starts

Definition

Delay that occurs when a platform must initialize a new runtime before handling a request. It is common in serverless and scale-to-zero environments.

Implementation / Usage Types

- Runtime initialization: load process, dependencies, and application code.
- Provisioned capacity: keep instances warm to reduce startup delay.
- Smaller package: reduce deployment and initialization work.
- Lazy initialization: defer nonessential startup work until needed.

Real-World Example

A serverless image-processing endpoint receives occasional requests. The first request after inactivity is slower because the runtime must initialize and load libraries.

Interview Follow-Up Questions

- When is a cold start acceptable?
- How do you measure cold-start contribution to P99?
- When should provisioned capacity be used?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

90. Serverless Limits

Definition

Constraints imposed by function/serverless platforms, such as maximum execution time, memory, concurrency, package size, temporary storage, and event payload size.

Implementation / Usage Types

- Execution limit: maximum runtime per invocation.
- Concurrency limit: maximum simultaneous executions.
- Memory/CPU limit: bounded resources per invocation.
- Payload/storage limit: constraints on request size and temporary disk.

Real-World Example

A serverless report generator cannot finish a 20-minute job within the function timeout. The design changes to queue-based chunked processing with multiple short-lived workers.

Interview Follow-Up Questions

- Which workloads fit serverless well?
- How do limits influence architecture?
- How should long-running jobs be redesigned?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

91. Latency

Definition

Measures how long an operation takes from a defined start to a defined completion point. It is a user-experience and capacity signal.

Implementation / Usage Types

- Network latency: time spent traversing network paths.
- Application latency: processing time inside services.
- Database latency: time spent waiting for queries.
- End-to-end latency: total user-visible request time.

Real-World Example

A checkout request has 900 ms end-to-end latency. Tracing shows 500 ms in database calls, 250 ms in payment, and the rest in application and network overhead.

Interview Follow-Up Questions

- Where should latency be measured?
- How do you set a latency target?
- Why can average latency hide customer pain?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

92. Throughput

Definition

Measures how much work a system completes per unit time, such as requests/second, messages/second, or transactions/minute.

Implementation / Usage Types

- Request throughput: completed requests per second.
- Data throughput: bytes or records processed per second.
- Transaction throughput: successful business operations per second.
- Consumer throughput: messages processed per second.

Real-World Example

A payment processor handles 2,000 transactions/second at peak. Capacity planning compares this demand with the sustainable throughput of each downstream dependency.

Interview Follow-Up Questions

- What limits throughput first?
- How do batching and parallelism affect throughput?
- Why can higher throughput increase latency?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

93. P99 Latency

Definition

The 99th percentile latency is the value below which 99% of measured requests complete. It focuses attention on the slowest 1% of requests.

Implementation / Usage Types

- Rolling P99: percentile over a time window.
- Endpoint P99: measure separately for each API.
- Segmented P99: compare region, customer tier, or operation.
- Histogram-derived P99: estimate from latency buckets.

Real-World Example

If API latency is P99=1.8 seconds, 99% of requests are at or below 1.8 seconds in the selected measurement window; the remaining 1% are slower.

Interview Follow-Up Questions

- Why not use only average latency?
- How does sample size affect percentile confidence?
- What could make P99 spike while average stays stable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

94. Tail Latency

Definition

Refers to unusually slow requests in the high percentile range. In distributed systems, tail latency can be amplified when a request waits on several downstream operations.

Implementation / Usage Types

- High-percentile latency: P95/P99/P99.9 behavior.
- Fan-out amplification: one request waits for multiple dependencies.
- Queueing tail: overloaded workers create long waits.
- GC/CPU tail: runtime pauses or contention affect a small fraction of requests.

Real-World Example

A page calls ten backend services in parallel. Even if each has a 99% chance of being fast, the chance that at least one is slow can become significant.

Interview Follow-Up Questions

- Why does fan-out amplify tail latency?
- How do hedged requests help and when can they hurt?
- How do you budget latency across dependencies?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

95. Network Partitions

Definition

A network partition occurs when nodes that should communicate cannot reliably exchange messages. Distributed systems must define how they behave when communication is broken.

Implementation / Usage Types

- Hard partition: communication is completely unavailable.
- Partial partition: some nodes can communicate while others cannot.
- Packet loss/latency partition: messages arrive slowly or unreliably.
- Partition-aware quorum: require enough members before accepting certain operations.

Real-World Example

A three-node cluster loses communication between one node and the other two. The quorum-based design allows the two-node side to continue while the isolated node stops accepting authoritative writes.

Interview Follow-Up Questions

- How does partition behavior relate to CAP?
- What is split brain?
- How should clients react during a partition?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

96. Clock Skew

Definition

Clock skew is the difference between clocks on machines. Distributed systems cannot safely assume all machines have exactly the same time.

Implementation / Usage Types

- NTP synchronization: periodically correct system clocks.
- Logical clocks: track ordering without relying on wall-clock time.
- Monotonic clocks: measure durations without being affected by wall-clock adjustments.
- Clock-bound leases: use bounded clock assumptions carefully.

Real-World Example

A distributed lock uses a TTL. Because clock differences can exist, the design also uses ownership tokens and fencing rather than assuming a perfectly synchronized clock.

Interview Follow-Up Questions

- Why should elapsed time use monotonic clocks?
- How can clock skew break token expiration?
- When are logical clocks preferable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

97. DNS

Definition

Domain Name System maps names to records such as IP addresses and service endpoints. DNS also provides caching and a distributed control plane for routing.

Implementation / Usage Types

- A/AAAA records: map names to IPv4/IPv6 addresses.
- CNAME: aliases one name to another.
- SRV: publishes service/port information.
- DNS-based traffic routing: choose endpoints based on geography or health in supported systems.

Real-World Example

A public API uses api.example.com. DNS points clients toward the appropriate edge or load-balancing layer, while TTL controls how long resolvers cache the result.

Interview Follow-Up Questions

- What does DNS TTL actually control?
- Why can DNS failover take time?
- How does DNS differ from service discovery inside a cluster?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

98. TCP vs UDP

Definition

TCP provides connection-oriented, reliable, ordered byte-stream delivery. UDP provides connectionless datagrams without built-in reliability or ordering.

Implementation / Usage Types

- TCP: reliable ordered transport for HTTP/1.1, HTTP/2, many databases, and other protocols.
- UDP: low-overhead datagrams where the application handles reliability if needed.
- QUIC: reliable streams and transport features built over UDP.
- Hybrid application behavior: choose based on latency, ordering, loss tolerance, and protocol needs.

Real-World Example

A traditional HTTPS connection may use TCP plus TLS, while HTTP/3 uses QUIC over UDP to improve connection establishment and multiplexing behavior.

Interview Follow-Up Questions

- Why does HTTP/3 use UDP?
- What does TCP retransmission cost under packet loss?
- When is UDP appropriate for an application?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

99. HTTP/2 & HTTP/3

Definition

HTTP/2 improves HTTP/1.1 with binary framing, multiplexed streams, header compression, and better connection utilization. HTTP/3 maps HTTP semantics onto QUIC over UDP.

Implementation / Usage Types

- HTTP/2 multiplexing: many streams share a connection.
- HPACK/QPACK: compress HTTP headers.
- HTTP/3/QUIC: transport with streams, encryption, and improved connection migration behavior.
- Protocol negotiation: clients and servers select supported versions.

Real-World Example

A web application serves many small resources. HTTP/2 reduces connection overhead through multiplexing; HTTP/3 can improve behavior on lossy networks because QUIC avoids TCP-level head-of-line blocking across independent streams.

Interview Follow-Up Questions

- What problem does HTTP/2 solve versus HTTP/1.1?
- How does HTTP/3 differ from HTTP/2 at the transport layer?
- What infrastructure must support HTTP/3?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

100. gRPC

Definition

gRPC is an RPC framework commonly using Protocol Buffers and HTTP/2. It supports strongly typed service contracts, streaming, deadlines, and efficient binary serialization.

Implementation / Usage Types

- Unary RPC: one request and one response.
- Server streaming: one request produces a stream of responses.
- Client streaming: client sends a stream and receives a response.
- Bidirectional streaming: both sides send streams concurrently.

Real-World Example

An internal order service calls inventory through gRPC with a strict deadline and typed protobuf contract, reducing ambiguity between independently deployed services.

Interview Follow-Up Questions

- When is gRPC preferable to REST?
- How do you evolve protobuf schemas safely?
- How do you expose gRPC services to browsers?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

101. Webhooks

Definition

Webhooks are HTTP callbacks sent by one system to another when an event occurs. They let consumers receive notifications without repeatedly polling the producer.

Implementation / Usage Types

- Event webhook: notify when a business event occurs.
- Signed webhook: include a signature so the receiver can verify origin.
- Retrying webhook: producer retries failed deliveries.
- Batch webhook: combine multiple events into one delivery where appropriate.

Real-World Example

A payment provider sends a signed payment_succeeded webhook to a merchant backend. The merchant verifies the signature and processes the event idempotently.

Interview Follow-Up Questions

- How do you authenticate webhooks?
- How do you handle duplicate delivery?
- What retry policy should a webhook provider use?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

102. API Versioning

Definition

Allows an API contract to evolve while controlling compatibility for existing clients.

Implementation / Usage Types

- URI versioning: /v1/orders and /v2/orders.
- Header versioning: version specified through a request header.
- Query versioning: version expressed as a query parameter.
- Content negotiation: media type/version in the Accept header.

Real-World Example

A public partner API keeps v1 available while v2 introduces a changed response model. Deprecation timelines and migration documentation give partners time to move.

Interview Follow-Up Questions

- When should an API get a new major version?
- How long should old versions be supported?
- What changes are backward compatible?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

103. Semantic Versioning

Definition

Semantic Versioning uses MAJOR.MINOR.PATCH to communicate compatibility expectations for software releases.

Implementation / Usage Types

- MAJOR: incompatible API changes.
- MINOR: backward-compatible functionality additions.
- PATCH: backward-compatible bug fixes.
- Pre-release/build metadata: communicate versions before stable release and attach build information.

Real-World Example

A shared SDK moves from 2.4.3 to 2.5.0 when adding a backward-compatible API, and to 3.0.0 when removing a supported method.

Interview Follow-Up Questions

- Does every internal service need semantic versioning?
- What qualifies as breaking?
- How should dependency ranges be selected?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

104. Infrastructure as Code

Definition

Defines infrastructure in version-controlled, repeatable configuration rather than manually creating resources through a console.

Implementation / Usage Types

- Declarative IaC: describe desired infrastructure state.
- Imperative automation: scripts execute steps to create resources.
- Configuration management: manage OS/software configuration.
- Policy as code: enforce infrastructure rules automatically.

Real-World Example

A platform team stores network, compute, database, and IAM definitions in Git. Pull requests review infrastructure changes before an automated pipeline applies them.

Interview Follow-Up Questions

- What should be stored in Git?
- How do you manage secrets in IaC?
- How do you detect and reconcile manual drift?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

105. Terraform

Definition

Terraform is an infrastructure-as-code tool that describes desired resources using configuration and manages their lifecycle through providers and state.

Implementation / Usage Types

- Resource provisioning: define cloud/network/database resources.
- Modules: package reusable infrastructure patterns.
- Remote state: share state safely across a team.
- Plan/apply workflow: review intended changes before execution.

Real-World Example

A team defines an AWS VPC, RDS cluster, security groups, and application infrastructure as Terraform modules. CI runs plan on pull requests and controlled apply after approval.

Interview Follow-Up Questions

- Why is state important?
- How do you handle state locking?
- How do you prevent accidental destructive changes?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

106. Helm Charts

Definition

Helm packages Kubernetes manifests into reusable charts with templates and configurable values. It helps standardize application deployment to Kubernetes.

Implementation / Usage Types

- Chart: package containing templates and metadata.
- Values: environment-specific configuration inputs.
- Release: an installed version of a chart in a cluster.
- Library chart: reusable templates shared across application charts.

Real-World Example

A platform team creates a standard API chart containing Deployment, Service, probes, resources, and autoscaling templates. Teams provide only service-specific values.

Interview Follow-Up Questions

- How do you manage secrets with Helm?
- How do you test chart changes?
- When would raw manifests or another deployment tool be preferable?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

107. Build Caching

Definition

Reuses outputs from unchanged build steps so CI or local builds complete faster and consume fewer compute resources.

Implementation / Usage Types

- Dependency cache: reuse downloaded packages.
- Layer cache: reuse unchanged container build layers.
- Compiler cache: reuse compiled artifacts where safe.
- Remote cache: share build outputs across CI runners or developers.

Real-World Example

A Docker build installs dependencies in a stable layer before copying frequently changing source code. Dependency installation is reused when package manifests have not changed.

Interview Follow-Up Questions

- What makes a cache key safe?
- When can stale build artifacts be dangerous?
- How do you measure cache effectiveness?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

108. Dependency Hell

Definition

Describes conflicts, incompatibilities, security issues, or upgrade complexity caused by a large or poorly managed dependency graph.

Implementation / Usage Types

- Version conflict: two libraries require incompatible versions.
- Transitive dependency problem: an indirect dependency introduces risk.
- Breaking upgrade: a major version changes behavior.
- Supply-chain risk: compromised or malicious packages enter the graph.

Real-World Example

A Node.js service depends on two libraries that require incompatible versions of the same package. The team upgrades one library, pins safe versions, and verifies the lockfile.

Interview Follow-Up Questions

- How do lockfiles help?
- How should dependency upgrades be governed?
- How do you balance security patches against breaking-change risk?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

109. Production Incidents

Definition

An incident is an unplanned event that materially affects customers, service reliability, security, or business operations and requires coordinated response.

Implementation / Usage Types

- Degradation: service works but violates expected performance/reliability.
- Outage: significant loss of service.
- Data incident: corruption, loss, or incorrect business state.
- Security incident: unauthorized access or malicious activity.

Real-World Example

A deployment causes checkout failures to rise sharply. Incident response pauses further rollout, identifies the failing dependency, restores service, communicates impact, and captures follow-up actions.

Interview Follow-Up Questions

- Who is the incident commander?
- What should happen in the first 15 minutes?
- How do you separate mitigation from root-cause investigation?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

110. On-Call

Definition

On-call is an operational responsibility where designated engineers respond to service alerts and incidents during defined coverage periods.

Implementation / Usage Types

- Primary/secondary rotation: primary responds first; secondary provides backup.
- Follow-the-sun: hand off coverage across regions/time zones.
- Escalation policy: page another team when ownership or severity requires it.
- Runbook-driven response: documented steps reduce dependence on individual memory.

Real-World Example

A critical payments service has primary and secondary engineers with escalation after a defined response window. Alerts include service context and links to runbooks.

Interview Follow-Up Questions

- What makes an on-call rotation sustainable?
- Which alerts should page an engineer?
- How do you measure operational load?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.

111. Postmortems

Definition

A structured review after an incident that explains what happened, customer impact, contributing factors, detection and response, and prevention actions. A good postmortem focuses on learning rather than blame.

Implementation / Usage Types

- Blameless postmortem: analyze system conditions rather than individual fault.
- Timeline: reconstruct events and decisions in order.
- Root/contributing factors: distinguish immediate trigger from systemic conditions.
- Corrective actions: assign owners and measurable due dates.

Real-World Example

After a database outage, the team documents the timeline, why failover was delayed, what alerts missed the condition, and which engineering/process changes will prevent recurrence.

Interview Follow-Up Questions

- How do you avoid a postmortem becoming a blame document?
- Which actions are highest priority?
- How do you verify that corrective actions actually reduced risk?

Interview lens	What to discuss
Trade-off	Cost vs. performance, consistency vs. availability, simplicity vs. scale.
Failure mode	What breaks first, how it is detected, and what the fallback is.
Measurement	Latency, throughput, errors, saturation, reliability, or business KPI.